

Development of a RISC-V based GPGPU

^{1st} Sebastian Hernandez Sandoval
School of electronics engineering
Costa Rica Institute of Technology
Cartago, Costa Rica
s.hernandez.1@estudiantec.cr

^{2nd} David Medina Mayorga
School of electronics engineering
Costa Rica Institute of Technology
Cartago, Costa Rica
david.medina@estudiantec.cr

^{3rd} Tiler Ureña Vargas
School of electronics engineering
Costa Rica Institute of Technology
Cartago, Costa Rica
tayurena7@estudiantec.cr

^{4th} Jia Chang Zou Mo
School of electronics engineering
Costa Rica Institute of Technology
Cartago, Costa Rica
jiachangzm@estudiantec.cr

Abstract—This study explores open-source GPGPU designs based on the RISC-V instruction set architecture (ISA). Existing solutions, such as Vortex and Ventus, have demonstrated the feasibility of executing SIMT on open-hardware platforms; however, limitations remain in terms of software portability, end-to-end compilation workflows, and architectural flexibility. This paper reviews the current state of the art and identifies the key challenges involved in developing a RISC-V GPU capable of executing PTX and HIP/ROCm workloads. To address these identified gaps, the paper proposes research directions and an architectural framework for future development.

Index Terms—GPGPU, RISC-V

I. INTRODUCTION

General-purpose graphics processing units (GPGPUs) are a fundamental topic in current high-performance computing as they can be used in machine learning workloads, scientific simulation, and computer vision. The GPGPUs use a Single Instruction Multiple-Thread (SIMT) model that allows thousands of lightweight threads to be processed simultaneously within a shared instruction stream. This allows a higher data parallelism and competitive energy efficiency systems [1]. With the continuous growth of the demand for accelerated computing, mostly driven by language models and Generative AI, the need for flexible, open, and research-accessible GPU platforms also grows [2].

Even with all the demand that the GPGPUs have now, the publicly available GPU infrastructure is indeed quite limited. Some commercial GPU architectures, like the ones from NVIDIA and AMD, are bound to a specific and proprietary ISA, toolchains, and programming models. One of the most prominent examples is NVIDIA's CUDA ecosystem and its Parallel Thread Execution (PTX) virtual ISA [3]. This concentrates hardware research around a single vendor's microarchitecture, and that definitely limits the community's ability to explore alternative designs, do new experiments, or do custom execution pipelines for new workloads.

The RISC-V instruction set architecture offers a promising path to follow. RISC-V follows a modular, extensible,

and royalty-free option, which makes it ideal for domain-specific hardware research, and its rapidly growing open-source ecosystem lowers the barrier to building complete hardware-software stacks. In recent work, it can be seen that small sets of custom RISC-V instructions can be enough to support SIMT [1], and that RVV, the RISC-V vector extension, can be used as the foundation for high-performance GPGPU microarchitectures [4]; however, there is a gap, there is not an existing open-source solution that provides an end-to-end, verified pipeline from PTX or ROCm binaries to native execution on a RISC-V-based GPU with hardware SIMT support.

This work addresses that gap by proposing the design of an open-source RISC-V GPGPU that, first, implements the SIMT programming model, so that the mapping from PTX constructs (threads, warps, cooperative thread arrays, grids) to hardware execution primitives can be done. Then, provide a compilation path that translates both CUDA and HIP/ROCm kernels to RISC-V machine code. This architecture provides support for integer and single-precision floating-point instructions, with compute units based on the RVV extension. Finally, this implementation targets synthesis and validation on an FPGA platform. The design and all evaluation artifacts follow open-source development practices to ensure full reproducibility.

The remainder of this paper is organized as follows. Section II does a quick look into the state of the art and also identifies some of the open challenges and gaps that motivate this work. Section III presents the proposed architecture. Section IV reports experimental results and analysis. Section ?? discusses the implications and compares the solution. Finally, Section VI reviews the main findings and defines future work.

II. BACKGROUND AND RELATED WORK

A. The SIMT Execution Model

GPUs achieve high throughput by running thousands of lightweight threads concurrently. To make this practical in hardware, threads are grouped into fixed-width bundles called warps, where all threads in the same warp execute the same

instruction at the same time, each on its own data lane. This model is known as Single Instruction Multiple Threads (SIMT). A central hardware component, the warp scheduler, maintains a separate program counter and an active mask for each warp, allowing independent warps to interleave on the same pipeline to hide memory latency.

The main efficiency challenge arises when threads within a warp reach a conditional branch and take different paths; this situation is called branch divergence. Since all threads must share one instruction stream, the hardware must execute both branches sequentially, masking off the threads that did not take each path. The point where all threads eventually reunite is called the immediate post-dominator (IPDOM), and it is tracked with a per-warp stack. The more divergent the workload, the more cycles are wasted on masked-out lanes, this is why the divergence management is a fundamental concern in the design in any GPGPU microarchitecture.

B. Complete RISC-V GPGPU Platforms

Vortex [1], developed at Georgia Tech, is the most thoroughly studied open-source RISC-V GPU to date. Its central claim is that the standard RV32IMF scalar ISA requires only five additional instructions to fully support SIMT execution: `wspawn` to launch a set of warps, `tmc` to update which threads in a warp are active, `split/join` to enter and exit a divergent region, and `bar` to synchronize threads across a core. This approach keeps the processor pipeline close to a conventional five-to-seven stage in-order design, extended with a warp scheduler and an IPDOM stack for reconvergence. On the software side, Vortex exposes an OpenCL programming interface through the POCL open-source runtime.

Ventus [4], from Tsinghua University, takes a different design philosophy. Rather than keeping the ISA minimal and adding a handful of custom instructions, Ventus uses the RISC-V Vector Extension (RVV) as its native execution substrate: vector registers map directly to the SIMD lanes of a warp, allowing the compiler to express SIMT parallelism through standard vector operations. Custom instructions are added only for the GPU-specific operations that RVV does not cover: predicated branching, warp-level barriers, and tensor multiply-accumulate units.

C. Partial Solutions and Software Portability Frameworks

Several works address individual layers of the problem. CuPBoP [3] translates CUDA programs at the LLVM/NVVM IR level to non-NVIDIA backends, including x86, AArch64, and RISC-V CPUs, achieving broad CUDA kernel coverage without source-level changes. However, it targets CPUs and therefore lacks hardware SIMT support. Notably, AMD’s HIP/ROCm stack follows an analogous IR-level path through the AMDGPU LLVM backend, so the same translation strategy is applicable to ROCm kernels without additional host-side changes.

Han et al. [5] established an end-to-end pipeline for executing CUDA kernels on a RISC-V GPU, routing code from NVVM IR through SPIR-V to OpenCL IR via POCL

and running it on Vortex, with demonstrated support for multi-threading and atomic instructions. SoftCUDA [6] extends this to full end-to-end CUDA support on Vortex without source modification, but the host-side memory API coverage remains incomplete.

Skybox [7] delivers the first open-source Vulkan API implementation on RISC-V hardware, extending Vortex with a rasterization unit and introducing an LLVM transformation that inserts split/join instructions. In the machine-learning domain, Nada et al. [8] integrate a tensor core into Vortex with cooperative warp execution for GEMM, achieving up to $3\times$ speedup over the standard baseline.

Finally, FGPU [9] is an OpenCL-programmable soft-GPU for FPGAs based on a MIPS-derived ISA, delivering $49\times$ speedup over a MicroBlaze scalar baseline and demonstrating the viability of SIMT accelerators on reconfigurable fabric. Its non-RISC-V ISA, however, excludes it from the RISC-V compiler toolchain.

D. Open Challenges

The surveyed literature reveals four persistent gaps that motivate this work: (1) while existing solutions offer partial support for running PTX or ROCm on RISC-V GPUs, there are currently no experimentally validated, fully open-source, and fully functional unified solution; (2) CUDA/PTX host-side API coverage on open GPU hardware remains incomplete across all frameworks; (3) the design trade-off between minimal ISA extension (Vortex style) and RVV-native SIMT (Ventus style) under integer and floating-point workloads has not been evaluated on the FPGA targets available in academic settings; and (4) the faithful mapping of the PTX thread hierarchy (threads, warps, blocks, grids) to a RISC-V core-based substrate has not been systematically addressed in the context of a configurable open-source microarchitecture.

III. METHODS

A. Identified Challenges

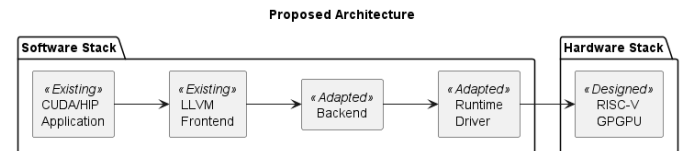


Fig. 1. Proposed Architecture

The state-of-the-art analysis reveals that, although several open-source RISC-V GPGPU architectures have been proposed, none of them fully addresses the requirements of a high-level hardware/software co-design methodology targeting FPGA platforms. These solutions typically focus on isolated aspects of the problem, either hardware or software, without providing an integrated development methodology.

From the comparative analysis, the following challenges have been identified:

- **C1. Limited hardware/software integration.** Existing implementations generally focus either on hardware architecture or software support, lacking a unified co-design methodology.
- **C2. Immature compiler and runtime support.** LLVM backends and runtime environments require adaptations to support custom architectural features and ISA extensions.
- **C3. RTL-centric development methodologies.** Most open-source GPGPUs are developed directly in RTL, limiting architectural exploration and increasing design complexity.
- **C4. Limited design-space exploration.** Existing architectures provide little flexibility to evaluate alternative scheduling, memory hierarchy, and execution strategies.
- **C5. FPGA implementation constraints.** Resource limitations require careful optimization of compute units, memory hierarchy, and synchronization mechanisms.

These challenges motivate the development of an integrated hardware/software co-design methodology capable of combining compiler adaptation, architectural modeling, high-level synthesis, and FPGA implementation within a unified development flow.

B. Proposed Solution

This work proposes a hardware/software co-design methodology for an open RISC-V GPGPU targeting FPGA platforms. Rather than developing every software component from scratch, the proposed solution reuses the existing LLVM ecosystem while adapting only those components required to support the proposed architecture.

Figure 2 illustrates the proposed hardware/software co-design methodology.

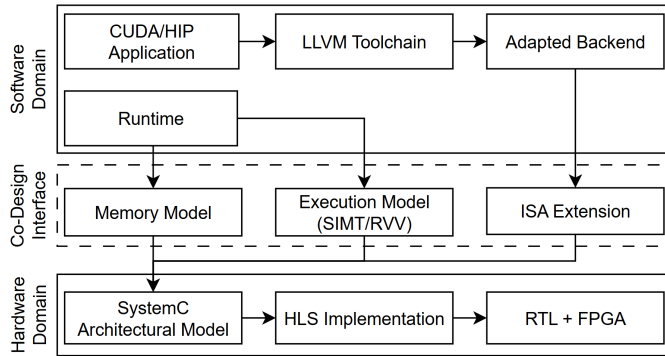


Fig. 2. Co-design methodology

At the software level, CUDA/HIP applications are compiled through the LLVM toolchain. An adapted backend generates binaries compatible with the proposed RISC-V GPGPU architecture, while the runtime and driver manage kernel execution and communication with the FPGA platform.

The interaction between software and hardware is defined through a common architectural contract composed of ISA

extensions, thread hierarchy, memory model, and synchronization mechanisms. This interface guarantees consistency between compiler-generated code and the underlying hardware implementation. On the hardware side, the architecture is modeled in SystemC to enable functional validation and architectural exploration. Once validated, the model is implemented using High-Level Synthesis (HLS), producing synthesizable RTL suitable for FPGA deployment.

This methodology enables rapid design iterations while maintaining consistency between software and hardware components, significantly reducing development effort compared to traditional RTL-first design approaches.

IV. EXPERIMENTAL PLAN

The proposed architecture will be evaluated using representative GPGPU workloads executed on the FPGA implementation, Figure 3. The objective is to compare different architectural configurations and quantify the impact of hardware design decisions on both performance and resource utilization.

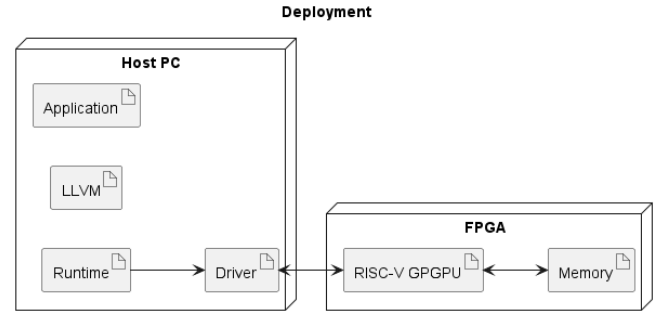


Fig. 3. Experiments methodology

A. Benchmark Selection

We select five kernels from the Rodinia 3.1 and PolyBench/GPU suites: vector addition (SAXPY), parallel reduction, dense matrix multiplication (GEMM), a 2D convolution kernel, and a heavily divergent kernel with nested conditional branches. Input data sizes range from 10^4 to 10^6 elements to evaluate scaling.

B. Design Space Exploration

Leveraging the configurable nature of our Ventus-based design, we explore the following parameters on the target FPGA: vector lane width (32, 64), number of clusters (1, 2, 4), and IPDOM stack enabled vs. disabled. A reduced set of configurations limits the total synthesis runs to twelve.

C. Metrics

For each configuration and kernel we record:

- **Resource utilization:** post-synthesis LUTs, flip-flops, BRAMs, and DSP slices reported by Vivado.
- **Maximum frequency** (F_{max}) in MHz.
- **Execution cycles** in simulation.
- **Estimated throughput** (GFLOP/s), calculated from F_{max} and cycle count.

- **Software coverage:** fraction of Rodinia kernels that compile and run correctly through the PTX to RVV path.
- **Divergence overhead:** ratio of active to total lanes in the divergent kernel.

D. Comparison Baselines

Results will be compared against the original Ventus [4] simulated in Verilator under identical memory latency assumptions and the Vortex GPGPU [1] synthesized on the same FPGA, where bitstreams are available.

CuPBoP [3] running on a RISC-V CPU for software compatibility metrics.

V. EXPECTED RESULTS AND ANALYSIS

The proposed methodology is expected to simplify the development of FPGA-based RISC-V GPGPUs by combining compiler adaptations with high-level hardware design techniques. The use of SystemC is expected to facilitate rapid architectural exploration before hardware synthesis, while HLS should reduce implementation effort compared with manual RTL development.

From the software perspective, adapting the LLVM backend and runtime is expected to enable the execution of representative CUDA/HIP workloads with minimal modifications to existing toolchains. From the hardware perspective, the FPGA implementation will provide quantitative information regarding resource utilization, operating frequency, and execution performance.

The experimental evaluation will allow the identification of architectural trade-offs between hardware complexity, FPGA resource consumption, and computational performance, validating the effectiveness of the proposed hardware/software co-design methodology.

VI. CONCLUSIONS

TBD

ACKNOWLEDGMENT

TBD

REFERENCES

- [1] F. Elsabbagh, B. Tine, P. Roshan, E. Lyons, E. Kim, D. E. Shim, L. Zhu, S. K. Lim, and H. Kim, "Vortex: OpenCL Compatible RISC-V GPGPU," *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2020.
- [2] J. Li et al., "RISC-V-Based GPGPU With Vector Capabilities for High-Performance Computing," *IEEE Trans. Syst., Man, Cybern.: Syst.*, 2025.
- [3] R. Han, J. Chen, B. Garg, X. Zhou, J. Lu, J. Young, J. Sim, and H. Kim, "CuPBoP: Making CUDA a Portable Language," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 4, Jun. 2024.
- [4] J. Li, K. Yang, C. Jin, X. Liu, Z. Yang, F. Yu, Y. Shi, M. Ma, L. Kong, J. Zhou, H. Wu, and H. He, "Ventus: A High-performance Open-source GPGPU Based on RISC-V and Its Vector Extension," in *Proc. IEEE 42nd Int. Conf. Computer Design (ICCD)*, Nov. 2024.
- [5] R. Han, B. Tine, J. Lee, J. Sim, and H. Kim, "Supporting CUDA for an Extended RISC-V GPU Architecture," *Workshop on RISC-V for Computer Architecture Research (CARRV)*, 2021.
- [6] C. Ahn, R. Han, U. Subramanya, J. Zhao, B. Tine, and H. Kim, "SoftCUDA: Running CUDA on Softcore GPU," *Proc. IEEE 33rd Annu. Int. Symp. Field-Programmable Custom Computing Machines (FCCM)*, 2025.
- [7] B. Tine, V. Saxena, S. Srivatsan, J. R. Simpson, F. Alzamar, L. Cooper, and H. Kim, "Skybox: Open-Source Graphic Rendering on Programmable RISC-V GPUs," *Proc. 28th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
- [8] A. Nada, G. M. Sarda, and E. Lenormand, "Cooperative Warp Execution in Tensor Core for RISC-V GPGPU," *Proc. IEEE Int. Symp. High Performance Computer Architecture (HPCA)*, 2025.
- [9] M. Al Kadi, B. Janssen, and M. Huebner, "General-Purpose Computing with Soft GPUs on FPGAs," *ACM Trans. Reconfigurable Technol. Syst.*, 2019.